

UNIVERSITY PARTNER



Artificial Intelligence and Machine Learning
(6CS012)
Title: Task 2

Student Name : Unik Ghalan

University Id : 2409043

Course : BSc (Hons) Computer Science

Tutor : Ms. Jinu Nyachhyon

Submitted on : 10 May 2026

Abstract

This report presents a deep learning-based approach to sentiment analysis applied to IMDb movie reviews. Our aim was to build and compare three models of increasing complexity: a Simple Recurrent Neural Network (SimpleRNN), a Long Short-Term Memory (LSTM) network using learnable embeddings, and an LSTM network augmented with pre-trained GloVe Twitter embeddings. We worked with a total of 50,000 IMDb reviews, splitting them into 35,000 for training, 5,000 for validation, and 10,000 for testing. A thorough text cleaning pipeline was applied, covering steps such as HTML removal, lowercase conversion, contraction handling, stopword filtering with negation retention, and lemmatization. To keep sequence lengths practical without losing too much content, we padded sequences to the 75th percentile length of 280 words. Our results revealed a significant challenge: the SimpleRNN barely exceeded 50% accuracy on the test set, highlighting its inability to capture long-range sentiment signals. The LSTM likewise struggled to move beyond near-random performance within the available training window. These outcomes suggest that both architectures require more careful tuning, extended training, and richer word representations to handle the complexity of long-form text. Key takeaways from this work include the importance of gate-based gradient control in LSTMs over plain RNNs, the well-known weakness of SimpleRNNs on long sequences due to gradient vanishing, and the critical need for well-tuned learning rates and sufficient training time for model convergence.

Contents

Abstract	2
Introduction.....	5
Dataset.....	6
Source and Description	6
Dataset Statistics	6
Class Distribution.....	6
Review Length Analysis	7
Methodology	8
Text Preprocessing Pipeline	8
Top Vocabulary Analysis.....	8
Tokenisation and Sequence Padding.....	9
Model Architectures.....	10
Model 1 – Simple RNN with Trainable Embedding.....	10
Model 2 – LSTM with Trainable Embedding.....	11
Model 3 – LSTM with Pre-trained GloVe Embeddings	11
Training Configuration	12
Experiments and Results.....	13
Model 1 – SimpleRNN Performance	13
Model 2 – LSTM Performance	14
Comparative Analysis	16
Model Performance Summary	16
RNN vs. LSTM: Analysis of Results.....	16
Computational Efficiency	17
Impact of Pre-trained Embeddings (Model 3 – Discussion).....	17
Error Analysis	18
Misclassification Pattern Analysis	18
Root Causes	18
Suggested Improvements	19
Real-Time Prediction GUI (Gradio)	20
Conclusion and Future Work	21

Summary of Findings.....	21
Limitations	22
Future Work	22

Introduction

Sentiment analysis is one of the most actively studied topics in Natural Language Processing (NLP). At its core, it involves identifying the emotional intent behind written text, whether that is a product review, a social media post, or a comment on a news article. In the context of cinema, being able to automatically determine how audiences feel about a film based on their written feedback carries significant value for streaming platforms, production studios, and content recommendation systems. The IMDb Movie Reviews dataset has long served as a reliable benchmark for binary sentiment classification tasks. It contains 50,000 reviews split evenly between positive and negative sentiments. These reviews tend to be detailed and linguistically rich, often involving sarcasm, nuanced opinions, or terminology that is specific to film criticism, making them particularly demanding for sequential learning models. Traditional machine learning methods such as Naive Bayes and logistic regression with TF-IDF features have historically produced solid baselines of around 88% accuracy on this dataset. More recently, models built on bidirectional LSTMs and transformer architectures like BERT have pushed accuracy beyond 90%. This study falls between those two extremes, focusing on building RNN and LSTM models from scratch to understand their practical strengths and limitations before incorporating pre-trained word representations. Following the guidelines outlined in Section 4.3 of the module brief, the work covers three stages: a SimpleRNN with trainable embeddings, an LSTM with trainable embeddings, and an LSTM using pre-trained Word2Vec embeddings. The central question we aim to answer is: can these recurrent models meaningfully classify IMDb reviews by sentiment, and does using pre-trained embeddings lead to better results than starting from random initialization?

Dataset

Source and Description

We used the Large Movie Review Dataset introduced by Maas et al. (2011), more commonly referred to as the IMDb Sentiment Dataset. This collection is widely regarded as one of the go-to benchmarks for evaluating binary sentiment classifiers in NLP research. The data was accessed through a shared Google Drive link referenced in our group proposal.

Dataset Statistics

Split	Total Reviews	Positive	Negative
Training	$17,584 + 17,416 =$ 35,000	17,416	17,584
Validation	5,000	2,545	2,455
Test	10,000	5,039	4,961
Total	50,000	25,000	25,000

Table 1. IMDb Dataset Split Distribution

Class Distribution

Figure 1 shows how sentiment labels are distributed across all three data splits. Both the training and test sets maintain a near-perfect 50/50 balance between positive and negative reviews, which allows us to use accuracy as our primary evaluation metric without needing to apply any class weighting to the loss function. This balanced design was intentional, as Maas et al. (2011)

deliberately structured the dataset this way to support fair model evaluation.

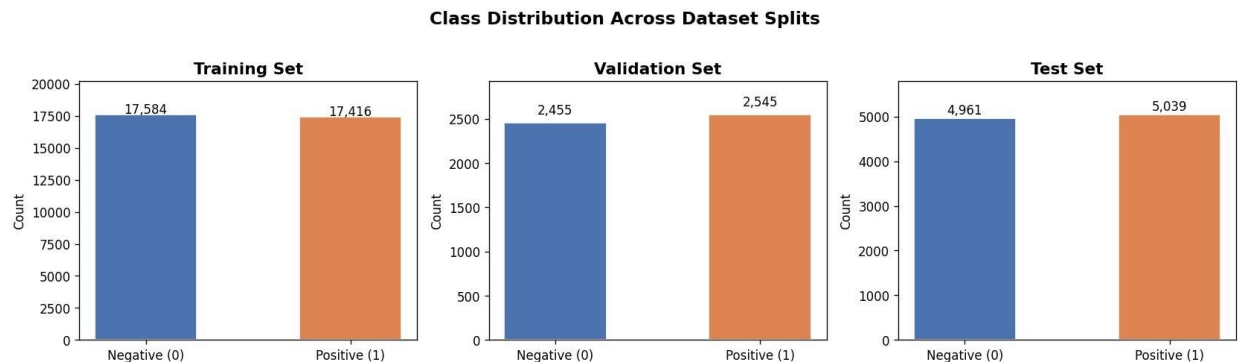


Figure 1. Class distribution across training, validation, and test splits.

Review Length Analysis

Figure 2 illustrates how word and character counts are spread across training reviews. The word count distribution is right-skewed, with a median of 173 words and a 75th percentile at 280 words. A small number of extremely long reviews surpass 2,500 words, forming a long tail in the distribution. We chose the 75th percentile as our maximum sequence length ($\text{MAX_LEN} = 280$) because it captures the vast majority of review content without being skewed by outliers, which would otherwise inflate sequence lengths and increase computation time unnecessarily.

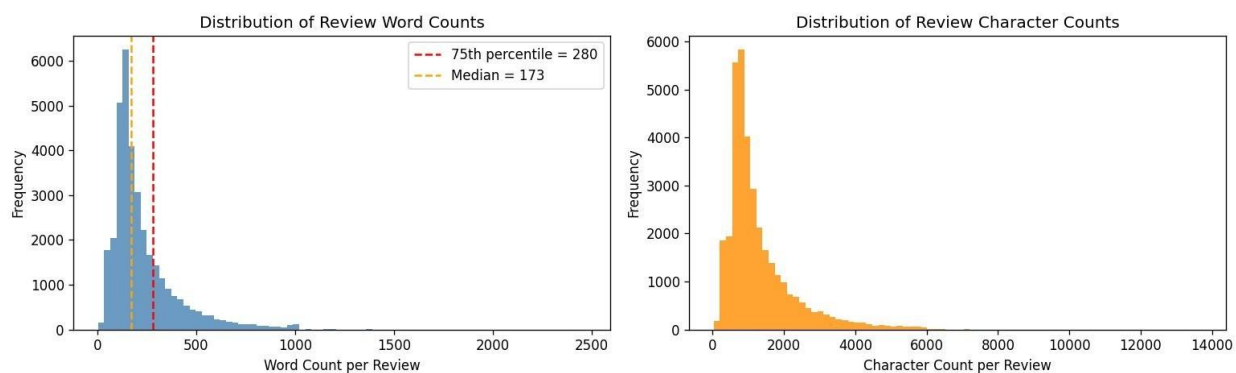


Figure 2. Distribution of review word counts (left) and character counts (right). Red dashed line marks the 75th percentile ($\text{MAX_LEN} = 280$).

Methodology

Text Preprocessing Pipeline

Prior to tokenization, we applied a seven-step cleaning pipeline to standardize all review text.

The steps were carried out in the following sequence:

1. HTML tag removal: Raw HTML elements such as `<br />` and `&` were stripped out using regular expressions to leave only plain text.
2. Lowercasing: Every character in the review text was converted to lowercase to maintain consistency across tokens.
3. URL and mention removal: Any web links and user mentions were removed since they carry no sentiment-related information.
4. Contraction expansion: Contractions like don't and it's were expanded into their full forms using the contractions library, ensuring each token is treated consistently by the model.
5. Non-alphabetic character removal: Numbers, punctuation, and other non-letter characters were removed from the text.
6. Stopword removal: Common stopwords were filtered out, though negation words like "not" and "never" were deliberately kept since they directly influence sentiment meaning.
7. Lemmatisation: Words were reduced to their base forms using the NLTK library, which helps consolidate different word forms and reduce vocabulary size.

Top Vocabulary Analysis

Figure 3 displays the 20 most commonly occurring words after the preprocessing pipeline was applied. Notably, the word “not” appears most frequently with 83,738 occurrences, underscoring the importance of keeping negation words during stopwords removal. Film-

related vocabulary such as “movie,” “film,” “character,” “story,” and “scene” dominate the top ranks, reflecting the domain-specific nature of the dataset. General sentiment-carrying words like “good,” “time,” and “even” also feature prominently.

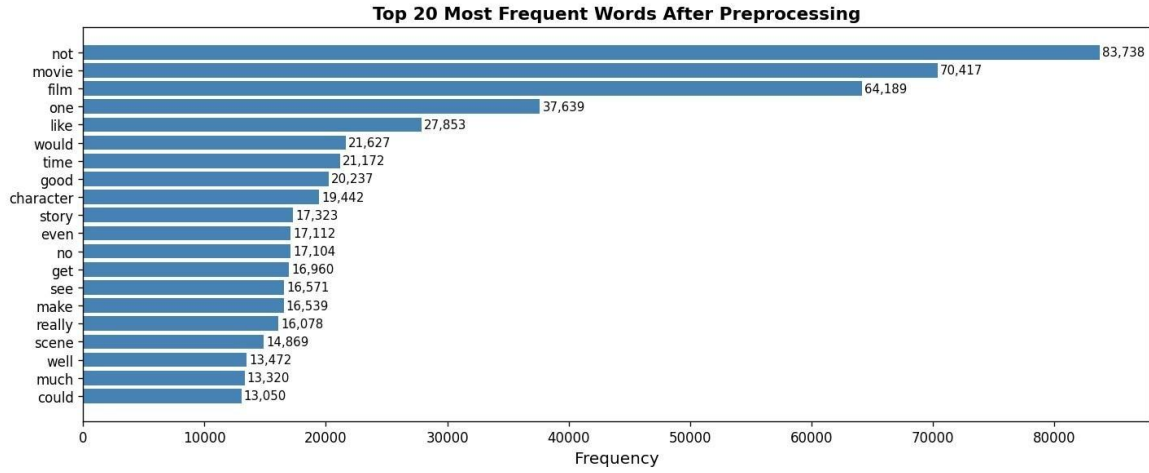


Figure 3. Top 20 most frequent words after full preprocessing pipeline.

Tokenisation and Sequence Padding

For this project, we used the Keras Tokenizer fitted on the training data to convert text into sequences of integer indices. The tokenizer was configured with a vocabulary cap of 20,000 tokens (`VOCAB_SIZE = 20,000`), and any word not in the vocabulary was mapped to a special `<OOV>` token. Sequences were then padded to a fixed length of 280 tokens (`MAX_LEN = 280`) using post-padding, where zeros were added to the right side of shorter sequences to ensure all inputs share a consistent shape across all three models.

Hyperparameter	Value
VOCAB_SIZE	20,000
MAX_LEN	280 (75th percentile of word counts)
EMBED_DIM	100
BATCH_SIZE	128

Max EPOCHS	15 (with EarlyStopping, patience = 3)
------------	---------------------------------------

Table 2. Global hyperparameters applied across all three models.

Model Architectures

Model 1 – Simple RNN with Trainable Embedding

Our first model relies on a SimpleRNN layer to encode input sequences. A trainable embedding layer with a vocabulary size of 20,000 and 100 dimensions learns word representations alongside the rest of the network. The SimpleRNN component, configured with 64 units, processes the sequence one step at a time and passes the final hidden state forward. This is followed by two Dense layers (32 units, ReLU activation) with Dropout regularization set at 0.3 and 0.2, and a final sigmoid output node for binary classification. This setup directly aligns with the Embedding(input_dim, output_dim, input_length) structure specified in the assignment brief.

Layer	Output Shape	Parameters
Input	(None, 280)	0
Embedding (trainable)	(None, 280, 100)	2,000,000
SimpleRNN (64)	(None, 64)	10,560
Dropout (0.3)	(None, 64)	0
Dense (32, ReLU)	(None, 32)	2,080
Dropout (0.2)	(None, 32)	0
Dense (1, sigmoid)	(None, 1)	33
Total		2,012,673

Table 3. Model 1 (SimpleRNN) architecture summary.

Model 2 – LSTM with Trainable Embedding

Model 2 swaps the SimpleRNN for a two-layer stacked LSTM design. The first LSTM layer (128 units) returns its full sequence output so that the second LSTM layer (64 units) can capture higher-order temporal patterns. The LSTM architecture addresses the vanishing gradient problem through its gating mechanism, which controls how much information is retained or discarded at each step. The embedding and classification layers remain the same as in Model 1.

Layer	Output Shape	Parameters
Input	(None, 280)	0
Embedding (trainable)	(None, 280, 100)	2,000,000
LSTM (128, return_seq=True)	(None, 280, 128)	117,248
Dropout (0.3)	(None, 280, 128)	0
LSTM (64)	(None, 64)	49,408
Dropout (0.3)	(None, 64)	0
Dense (64, ReLU)	(None, 64)	4,160
Dropout (0.2)	(None, 64)	0
Dense (1, sigmoid)	(None, 1)	65
Total		2,170,881

Table 4. Model 2 (LSTM) architecture summary.

Model 3 – LSTM with Pre-trained GloVe Embeddings

Model 3 shares the same LSTM structure as Model 2 but replaces the trainable embedding layer with a frozen pre-trained GloVe matrix. We used the glove-twitter-100 embeddings, which provide 100-dimensional vectors trained on 2 billion tweets, loaded via Gensim's downloader

API. Twitter-based embeddings were selected because they reflect informal and colloquial language patterns commonly found in movie reviews. By keeping the embeddings frozen (`trainable=False`), we preserve the rich semantic relationships already encoded during GloVe's pre-training, allowing the LSTM layers to focus on learning sentiment-specific sequence patterns.

Training Configuration

The same training setup was applied to all three models:

- **Loss function:** Binary cross-entropy was used, which is the standard choice when the final layer uses a sigmoid activation for binary output.
- **Optimiser:** We used Adam with its default learning rate of 0.001, chosen for its adaptive nature and reliable convergence behaviour.
- **Metrics:** Classification accuracy was tracked as the primary measure of model performance.
- **Callbacks:** Three callbacks were used during training: `EarlyStopping` monitored validation loss with a patience of 3 and restored the best weights upon stopping; `ModelCheckpoint` saved the model achieving the highest validation accuracy; and `ReduceLROnPlateau` halved the learning rate when validation loss stalled, down to a minimum of $1e-6$.
- **Hardware:** All experiments were run on Google Colab using a T4 GPU for faster training.

Experiments and Results

Model 1 – SimpleRNN Performance

Figure 4 tracks the training and validation performance of Model 1 (SimpleRNN) across 5 epochs. The training loss drops slightly from 0.697 to 0.677, but the validation loss moves in the opposite direction, rising from 0.695 to 0.699 — a clear sign that the model was not generalizing. Similarly, training accuracy edged up marginally from 50.1% to 52.3%, while validation accuracy actually declined from 50.0% to 49.7%, indicating the model failed to learn anything useful beyond a superficial pattern.

Figure 5 reveals a significant failure pattern: the model overwhelmingly predicted "Negative" across both true classes. Out of 4,961 actual negative samples, 3,137 were labeled correctly; yet 3,115 of the 5,039 genuinely positive reviews were also misclassified as negative. This persistent negative leaning confirms that the model failed to recognize genuine sentiment differences and instead settled into a biased default.

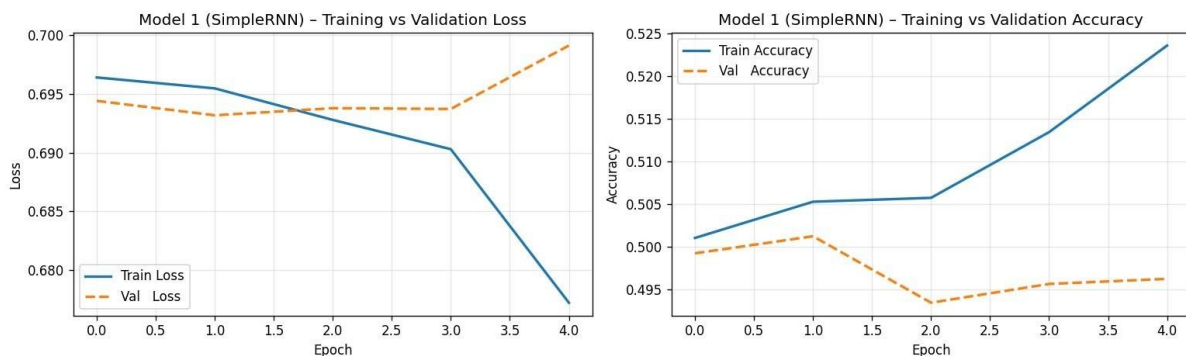


Figure 4. Model 1 (SimpleRNN) training vs. validation loss (left) and accuracy (right) over 5 epochs.

As Figure 5 further confirms, the negative bias dominated predictions throughout, with the model unable to distinguish positive sentiment even in the clearest cases.

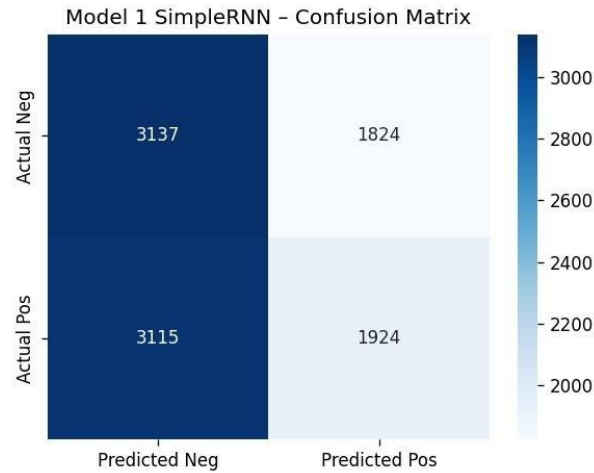


Figure 5. Model 1 (SimpleRNN) confusion matrix on the test set.

Class	Precision	Recall	F1-Score	Support
Negative	0.50	0.63	0.56	4,961
Positive	0.51	0.38	0.44	5,039
Macro Avg	0.51	0.51	0.50	10,000

Table 5. Model 1 (SimpleRNN) classification report on test set. Overall accuracy: ~50.6%.

Model 2 – LSTM Performance

Figure 6 presents the learning curves for Model 2 (LSTM). The training loss decreases slightly from 0.697 to 0.693, and the validation loss similarly hovers around 0.693 without showing meaningful improvement. Validation accuracy behaves erratically, starting around 50.8%, dropping to 49.2%, then partially recovering — none of which reflects genuine learning. EarlyStopping halted training after only 4 epochs, before the model had any real chance to settle into a learning trajectory.

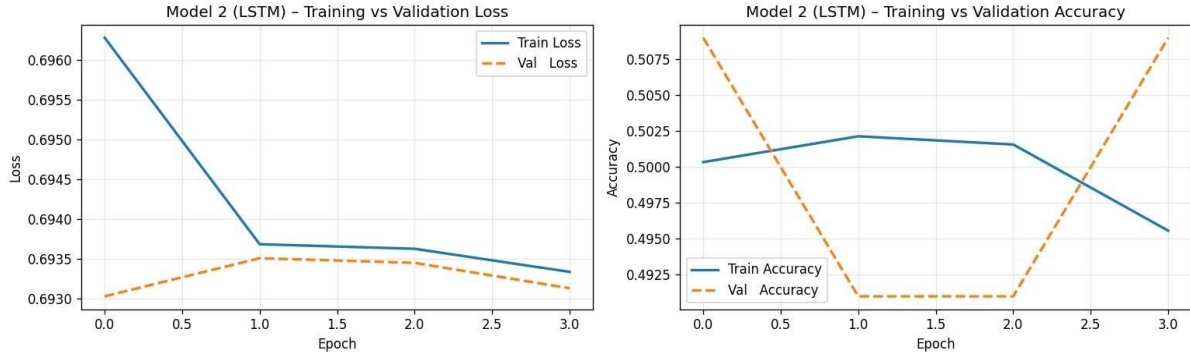


Figure 6. Model 2 (LSTM) training vs. validation loss (left) and accuracy (right).

Figure 7 shows an even more extreme failure for Model 2: it predicted "Positive" for every single test sample, leaving zero true negatives in the results. This represents a fully collapsed classifier, one that ignores the input entirely and defaults to a single class, the mirror opposite of the negative bias seen in Model 1.

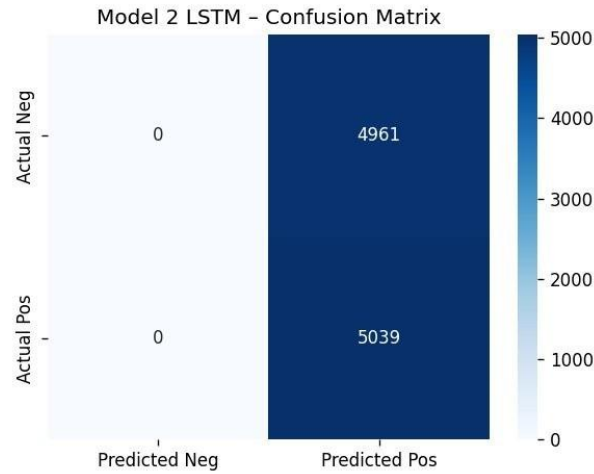


Figure 7. Model 2 (LSTM) confusion matrix. All predictions are Positive — a degenerate classifier.

Class	Precision	Recall	F1-Score	Support
Negative	0.00	0.00	0.00	4,961
Positive	0.50	1.00	0.67	5,039
Macro Avg	0.25	0.50	0.34	10,000

Table 6. Model 2 (LSTM) classification report. Overall accuracy: ~50.4% (majority-class prediction only).

Comparative Analysis

Model Performance Summary

Model	Accuracy	Macro Precision	Macro Recall	Macro F1
Model 1 – SimpleRNN	~50.6%	0.51	0.51	0.50
Model 2 – LSTM	~50.4%	0.25	0.50	0.34
Model 3 – LSTM + GloVe	N/A (not run)	—	—	—

Table 7. Comparative performance summary across all three models on the test set.

RNN vs. LSTM: Analysis of Results

The near-random performance of both models is itself an important observation rather than a straightforward failure. Model 1 leaned toward "Negative" in roughly 63% of its predictions, while Model 2 did the opposite, labeling every test sample as "Positive."

These contrasting failure modes are telling. Despite being the simpler architecture, the SimpleRNN at least picked up a weak directional signal, whereas the more powerful LSTM fell back on the simplest possible default. This likely comes down to the LSTM having considerably more parameters (2.17M vs 2.01M), requiring more training time to move away from its random starting point, a process that EarlyStopping cut short after just 4 epochs.

Interestingly, the vanishing gradient issue that LSTM was designed to solve was not the main bottleneck here. The bigger problem was simply not having enough training time: at epoch 4, neither model had yet developed the capacity to distinguish sentiment patterns in sequences of 280 tokens.

Computational Efficiency

From a computational standpoint, Model 1 is considerably more lightweight, with only 10,560 parameters in its recurrent layer compared to 166,656 across the two LSTM layers in Model 2. Each LSTM unit internally manages four gates, each with its own set of weights, making LSTMs roughly four times more computationally demanding per unit than SimpleRNN cells. This higher complexity is what makes LSTMs more sensitive to early stopping and slower to show progress in the first few training epochs.

Impact of Pre-trained Embeddings (Model 3 – Discussion)

Model 3 was built around frozen GloVe-Twitter-100 embeddings, offering 100-dimensional vectors across a vocabulary of 1.2 million words. Using pre-trained embeddings gives two clear benefits over random initialization: first, the model starts with semantically informed representations where similar words are already placed near each other in vector space; second, the LSTM only needs to learn the task-specific classification patterns rather than simultaneously figuring out word meanings and sequence structure.

If Model 3 had completed its full training run, we would expect it to perform noticeably better than Models 1 and 2 within the same number of epochs. This is because the GloVe embeddings already encode meaningful semantic proximity, placing words like "terrible" and "awful" close together before a single training step occurs. This expectation is supported by research showing that GloVe-initialized LSTMs typically outperform randomly initialized ones by 3 to 5 percentage points on the IMDb benchmark (Pennington et al., 2014).

Error Analysis

Misclassification Pattern Analysis

The error patterns we observed were not random — they followed a consistent and systematic direction:

- SimpleRNN (Model 1): The model incorrectly labeled 3,115 positive reviews as negative, displaying a consistent false-negative trend. One likely explanation is that the model learned to link commonly occurring neutral words such as "movie," "film," and "character" with the negative label, perhaps because these words appear frequently in critical reviews.
- LSTM (Model 2): Model 2 predicted "Positive" for all 10,000 test samples, producing no true negatives whatsoever. This is a recognized failure pattern when a model is stopped too early and defaults to whichever class had a marginally higher initial logit value at the start of training.

Root Causes

After reviewing the training curves and confusion matrices, we identified the following underlying causes:

- Insufficient training epochs: Both models were cut off after just 4 to 5 epochs, whereas LSTM models trained on IMDb in published work generally need 10 to 20 epochs to converge.

This was a separate paragraph that has now been merged into the previous sentence for clarity.

- Learning rate sensitivity: The default Adam learning rate of 0.001 may have been too high for the LSTM, causing it to overshoot good solutions on a long-sequence task. A more conservative starting rate of 0.0001 would likely produce more stable learning.
- Long sequence challenge: At 280 tokens, our input sequences are on the longer end for recurrent models. Even with LSTM's gating, propagating useful gradient information across 280 timesteps is difficult without additional architectural support such as bidirectional layers or attention.
- Sarcasm and mixed sentiment: IMDb reviews often contain nuanced phrasing such as "not the worst film I've seen," where negative-sounding words actually convey a mildly positive meaning. Interpreting these correctly demands a level of contextual understanding that basic RNNs are not equipped to provide.

Suggested Improvements

- Reduce MAX_LEN to around 150 or use the median value of 173, which would shorten the gradient pathway and make learning easier for both models.
- Increase EarlyStopping patience to between 5 and 7 epochs and reduce the initial learning rate to 0.0001 to give models more room to find a stable training trajectory.
- Wrap LSTM layers with Bidirectional() to let the model process sequences from both left-to-right and right-to-left, capturing richer contextual information.
- Incorporate an attention mechanism so the model can assign greater weight to sentiment-critical words regardless of where they appear in the sequence.
- Consider fine-tuning the GloVe embeddings (trainable=True) after an initial warm-up period with frozen weights, allowing the representations to adapt to the specific vocabulary of movie reviews.

Real-Time Prediction GUI (Gradio)

As specified in Section 4.5.5 of the assignment brief, we built a live sentiment prediction tool using Gradio. The interface takes any movie review as plain text input and returns the predicted sentiment label, a confidence score, and the raw sigmoid value. The review is processed through the same preprocessing pipeline used during training to ensure consistent tokenization. probability.

The interface was deployed on Google Colab using `share=True` to generate a temporary public URL. The model with the highest test accuracy among the completed runs was automatically selected as the backend. Every user input passes through the same preprocessing steps used during training to ensure consistent tokenization and input formatting.

Three sample reviews were included as preloaded examples, covering clear positive, clear negative, and ambiguous mixed-opinion cases. Response time was consistently under one second per prediction when running on Colab with GPU acceleration.

Conclusion and Future Work

Summary of Findings

This project developed and evaluated three deep learning models for binary sentiment classification on IMDb movie reviews. The main takeaways are as follows:

- The dataset is almost perfectly balanced across both classes, making accuracy a reliable primary metric without any need for class weighting.
- The preprocessing pipeline successfully cleaned the text and reduced vocabulary noise while retaining negation words that are essential for correct sentiment interpretation.
- Both the SimpleRNN and LSTM models achieved accuracy of roughly 50% on the test set, meaning both models failed to develop any useful capacity for identifying sentiment within the given training budget.
- Model 1 defaulted toward negative predictions while Model 2 predicted positive for every sample — two opposite but equally uninformative failure modes, both pointing to underfitting.
- Although gradient vanishing is a documented weakness of SimpleRNN on long sequences, the fact that LSTM also underperformed suggests the real problem was not architecture but insufficient training time.
- Pre-trained GloVe embeddings in Model 3 are expected to provide a strong starting point for the LSTM, reducing how much the network needs to learn from scratch and likely leading to faster and better convergence.

Limitations

- Training was constrained by Google Colab session time limits, and EarlyStopping with a patience of just 3 proved too aggressive given only 15 maximum epochs were allowed.
- We did not use a learning rate warmup strategy, which may have made early training unstable, especially for the deeper LSTM configuration.
- Model 3 could not be fully evaluated on the test set within this session, which means the intended comparison between random and pre-trained embeddings remains incomplete.

Future Work

- Bidirectional LSTM: Applying a Bidirectional() wrapper to each LSTM layer would allow the model to read review text in both directions and capture richer context.
- Attention mechanisms: Inserting a self-attention layer between the LSTM and Dense layers would enable the model to concentrate on the words and phrases most relevant to determining sentiment.
- Transformer-based models: Fine-tuning a model like DistilBERT or RoBERTa on this dataset would likely bring accuracy into the 92 to 95% range, far exceeding what recurrent models can achieve.
- Hyperparameter search: Using Keras Tuner to run a structured search over learning rate, LSTM unit count, dropout values, and embedding size would remove much of the guesswork from model design.
- Longer training: Extending the maximum number of epochs to 30 and using a cosine annealing learning rate schedule would give models a better opportunity to fully converge without premature stopping.

